Machine Learning for Mechanical Engineers at CoEP Session 7: Sklearn Workflow

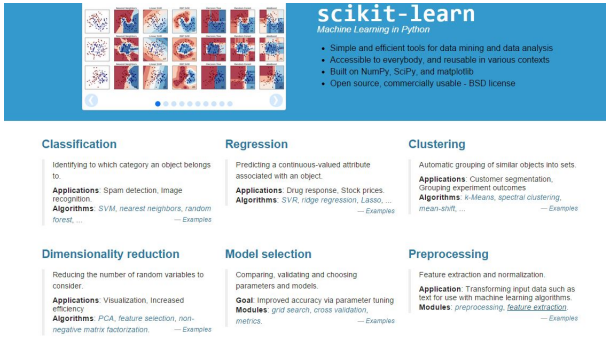
Session 7: Sklearn Workflow

Introduction to Scikit-Learn aka Sklearn

Scikit-Learn

- Open source machine learning library for Python.
- Features various classification, regression and clustering algorithms

Machine Learning in Python



Sklearn Paradigm

Scikit-Learn provides an object-oriented interface centered around the concept of an Estimator.

- An Estimator is any object that learns something from data.
- Classifiers, regressors, clusterers and preprocessors are all Estimators.
- Every Estimator learns through the same method, `fit()`.
- Models then predict with `predict()`, preprocessors transform with `transform()`.
- Swapping one algorithm for another changes one line, not the whole script.

Intuition

Every appliance in your house does a different job, but they all accept the same wall socket, so you never relearn the plug. In Scikit-Learn that socket is `fit`/`predict`. Learn it once and every algorithm in the library is already familiar.

Scikit-Learn's Estimator API

The steps in using the Scikit-Learn estimator API are as follows

- Choose a class of model by importing the appropriate estimator class from Scikit-Learn.
- Choose model hyper-parameters by instantiating this class.
- Arrange data into a features matrix and target vector.
- Fit the model to your data by calling the fit() method of the model instance.
  - For supervised learning, predict labels for unknown data using the predict().
  - For unsupervised learning, transform or infer properties of the data using transform() or predict().

Estimator

- `fit(X, y)` sets the state of the estimator from the training data.
- X is the feature matrix, a 2D numpy array of shape (n_samples, n_features).
- y is the target vector, a 1D array of shape (n_samples,) holding the responses.
- `predict(X)` returns the predicted class or value.

```
class Estimator(object):
    def fit(self, X, y=None):
        """Fits estimator to data. """
        # set state of ''self''
        return self
    def predict(self, X):
        """Predict response of ''X''. """
        # compute predictions ''pred''
        return pred
```

Sklearn: Installations

Scikit-Learn: Installation and Setup

Install the Anaconda distribution, then install the libraries used in this session.

```
conda install numpy scipy pandas matplotlib seaborn scikit
    -learn jupyter
```

Checking your installation

```
import numpy
print('numpy:', numpy.__version__)

import scipy
print('scipy:', scipy.__version__)

import pandas
print('pandas:', pandas.__version__)

import matplotlib
print('matplotlib:', matplotlib.__version__)

import sklearn
print('scikit-learn:', sklearn.__version__)

import seaborn
print('seaborn', seaborn.__version__)
```

Quick Check: The Estimator API

Think About It

A `StandardScaler` and a `LogisticRegression` are both Estimators, and both learn from the training data through `fit()`. Yet only one of them is later called with `predict()`. Which one is it, and what does the other one give you instead?

Quick Check: The Estimator API

Answer

Only `LogisticRegression` predicts, because it is a model: it learns a mapping from features to a target, so `predict(X)` returns an answer. `StandardScaler` is a preprocessor: it learns the mean and standard deviation of each column, so it offers `transform(X)`, which returns reshaped data rather than an answer. This is the whole Estimator API in one line: everything learns with `fit`, models then `predict`, preprocessors then `transform`.

Data Preprocessing with Scikit-Learn

Data Preprocessing for Pima-Indians Diabetis Dataset

(Ref: How To Prepare Your Data For Machine Learning in Python with Scikit-Learn - Jason Brownlee)

Read Data

Import Pima Inidans dataset and split into Features (X) and target (Y)

```
import pandas
import numpy

url = "https://raw.githubusercontent.com/jbrownlee/
     Datasets/master/pima-indians-diabetes.data.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', '
     pedi', 'age', 'class']

dataframe = pandas.read_csv(url, names=names)
array = dataframe.values

# separate array into input and output components
X = array[:,0:8]
Y = array[:,8]
```

Rescale Data

- Also called normalization: every attribute is rescaled into the range 0 to 1.
- This helps the optimization algorithms at the core of machine learning, such as gradient descent.
- It also helps algorithms that weight their inputs, such as regression and neural networks.
- It helps algorithms that use distance measures too, such as K-Nearest Neighbors.

Rescale Data

Rescale your data using scikit-learn using the MinMaxScaler class.

```
# Rescale data (between 0 and 1)
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler(feature_range=(0, 1))
rescaledX = scaler.fit_transform(X)

# summarize transformed data
numpy.set_printoptions(precision=3)
print(rescaledX[0:5,:])
```

Rescale Data

After rescaling you can see that all of the values are in the range between 0 and 1.

```
[[ 0.353  0.744  0.59   0.354  0.     0.501  0.234  0.483]
 [ 0.059  0.427  0.541  0.293  0.     0.396  0.117  0.167]
 [ 0.471  0.92   0.525  0.     0.     0.347  0.254  0.183]
 [ 0.059  0.447  0.541  0.232  0.111  0.419  0.038  0.   ]
 [ 0.     0.688  0.328  0.354  0.199  0.642  0.944  0.2
     ]]
```

Standardize Data

- Standardization transforms an attribute so that it has a mean of 0 and a standard deviation of 1.
- It suits attributes that already follow a Gaussian distribution, but with differing means and spreads.
- It suits techniques that assume Gaussian inputs, such as linear regression, logistic regression and linear discriminant analysis.

Intuition

Rescale (MinMaxScaler) and Standardize (StandardScaler) are easy to mix up: rescaling squeezes every value into a fixed $[0, 1]$ box regardless of shape, while standardizing centers the data on 0 and measures everything in "number of standard deviations": which is why standardized values can still go negative or exceed 1, unlike rescaled ones.

Standardize Data

Standardize data using scikit-learn with the StandardScaler class.

```
# Standardize data (0 mean, 1 stdev)
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler().fit(X)
rescaledX = scaler.transform(X)

# summarize transformed data
numpy.set_printoptions(precision=3)
print(rescaledX[0:5,:])
```

Standardize Data

The values for each attribute now have a mean value of 0 and a standard deviation of 1.

```
[[ 0.64   0.848  0.15   0.907 -0.693  0.204  0.468  1.426]
 [-0.845 -1.123 -0.161  0.531 -0.693 -0.684 -0.365 -0.191]
 [ 1.234  1.944 -0.264 -1.288 -0.693 -1.103  0.604 -0.106]
 [-0.845 -0.998 -0.161  0.155  0.123 -0.494 -0.921 -1.042]
 [-1.142  0.504 -1.505  0.907  0.766  1.41   5.485 -0.02
     ]]
```

Normalize Data

- Normalizing in scikit-learn refers to rescaling each observation (row) to have a length of 1 (called a unit norm in linear algebra).
- This preprocessing can be useful for sparse datasets (lots of zeros) with attributes of varying scales when using algorithms that weight input values such as neural networks and algorithms that use distance measures such as K-Nearest Neighbors.

Normalize Data

Normalize data in Python with scikit-learn using the Normalizer class.

```
# Normalize data (length of 1)
from sklearn.preprocessing import Normalizer

scaler = Normalizer().fit(X)
normalizedX = scaler.transform(X)
# summarize transformed data
numpy.set_printoptions(precision=3)
print(normalizedX[0:5,:])
```

Normalize Data

Each row is rescaled so that the sum of the squares of its values equals 1: the row's direction is preserved, only its length changes. A simple worked example, for one row of two features. Applied to every row of `X`, this is exactly what `normalizedX` above contains: each row now has unit length, so no single value in it can exceed 1 in magnitude.

```
row = [4, 3]                        # length = sqrt(4^2 +
     3^2) = 5
normalized_row = [4/5, 3/5]         # = [0.8, 0.6]
# check: 0.8^2 + 0.6^2 = 1.0
```

Binarize Data (Make Binary)

- You can transform your data using a binary threshold. All values above the threshold are marked 1 and all equal to or below are marked as 0.
- This is called binarizing your data or threshold your data. It can be useful when you have probabilities that you want to make crisp values.
- It is also useful when feature engineering and you want to add new features that indicate something meaningful.

Binarize Data

Create new binary attributes in Python using scikit-learn with the Binarizer class.

```
# binarization
from sklearn.preprocessing import Binarizer

binarizer = Binarizer(threshold=0.0).fit(X)
binaryX = binarizer.transform(X)
# summarize transformed data
numpy.set_printoptions(precision=3)
print(binaryX[0:5,:])
```

Binarize Data

You can see that all values equal or less than 0 are marked 0 and all of those above 0 are marked 1.

```
[[ 1.  1.  1.  1.  0.  1.  1.  1.]
 [ 1.  1.  1.  1.  0.  1.  1.  1.]
 [ 1.  1.  1.  0.  0.  1.  1.  1.]
 [ 1.  1.  1.  1.  1.  1.  1.  1.]
 [ 0.  1.  1.  1.  1.  1.  1.  1.]]
```

Putting It Together: Pipeline

- Scaling the whole dataset before splitting it leaks test information into training.
- A `Pipeline` chains preprocessing and model into one Estimator.
- It exposes the same `fit` and `predict` methods as any single model.
- Inside cross validation, the scaler is refitted on each training fold only.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = make_pipeline(StandardScaler(),
                     LogisticRegression(max_iter=1000))

# scaler is fitted per fold, never on the held-out data
pipe.fit(X, Y)
predictions = pipe.predict(X)
```

Quick Check: Data Preprocessing

Think About It

You fit a `StandardScaler` on your training data, then need to preprocess the test set before evaluating your model. Should you call `scaler.fit(X_test)` again, or reuse the scaler fitted on the training data?

Quick Check: Data Preprocessing

Answer

Always reuse the scaler fitted on the training data (`scaler.transform(X_test)`, no re-fit). Fitting a new scaler on the test set would compute a *different* mean/std from data the model has never seen, so the same raw value could be scaled differently between train and test, silently corrupting the features the model was trained on. Fitting only ever happens once, on training data; every other split reuses that same fitted scaler. A `Pipeline` enforces this for you.

Model Evaluation with Scikit-Learn

Classification Model Evaluation for Pima-Indians Diabetis Dataset

(Ref: Metrics To Evaluate Machine Learning Algorithms in Python - Jason Brownlee)

Read Data: the Same Pima Dataset

Same load step as the preprocessing section, repeated so this section stands on its own.

```
import pandas
import numpy

url = "https://raw.githubusercontent.com/jbrownlee/
     Datasets/master/pima-indians-diabetes.data.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', '
     pedi', 'age', 'class']

dataframe = pandas.read_csv(url, names=names)
array = dataframe.values

# separate array into input and output components
X = array[:,0:8]
Y = array[:,8]
```

Classification Accuracy

- Classification accuracy is the number of correct predictions as a ratio of all predictions made.
- This is the most common evaluation metric for classification problems, and also the most misused.
- It is only suitable when each class has roughly the same number of observations, which is rarely the case.
- It also assumes every kind of prediction error costs the same, which is often untrue.

Classification Accuracy

Accuracy score of approximately 77% accurate

```
from sklearn import model_selection
from sklearn.linear_model import LogisticRegression

seed = 7
kfold = model_selection.KFold(n_splits=10, shuffle=True,
     random_state=seed)
model = LogisticRegression(max_iter=1000)
scoring = 'accuracy'
results = model_selection.cross_val_score(model, X, Y, cv=
     kfold, scoring=scoring)
print("Accuracy: %.3f (%.3f)" % (results.mean(), results.
     std()))

# Output:
# Accuracy: 0.772 (0.050)
```

Logarithmic Loss

- Logarithmic loss (or logloss) is a performance metric for evaluating the predictions of probabilities of membership to a given class.
- The scalar probability between 0 and 1 can be seen as a measure of confidence for a prediction by an algorithm. Predictions that are correct or incorrect are rewarded or punished proportionally to the confidence of the prediction.

Logarithmic Loss

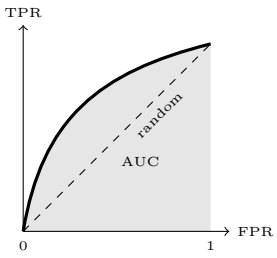
Smaller logloss is better with 0 representing a perfect logloss. As mentioned above, the measure is inverted to be ascending when using the cross_val_score() function.

```
seed = 7
kfold = model_selection.KFold(n_splits=10, shuffle=True,
     random_state=seed)
model = LogisticRegression(max_iter=1000)
scoring = 'neg_log_loss'
results = model_selection.cross_val_score(model, X, Y, cv=
     kfold, scoring=scoring)
print("Logloss: %.3f (%.3f)" % (results.mean(), results.
     std()))

# Output:
# Logloss: -0.485 (0.057)
```

Area Under ROC Curve

- Area under ROC Curve (AUC) measures how well a binary classifier separates positive cases from negative cases.
- An area of 1.0 is a perfect model, an area of 0.5 is no better than random.
- The curve trades off sensitivity (the true positive rate, also called recall) against specificity (the true negative rate).



The shaded area under the curve is the AUC. The dashed diagonal is a coin-flip model.

Intuition

AUC = 0.5 isn't "50% accurate": it means the model ranks positive and negative cases no better than a coin flip. AUC is threshold-free: it asks how often the model scores a random positive case above a random negative one. That is why it stays meaningful even when the classes are wildly imbalanced, unlike plain accuracy.

Area Under ROC Curve

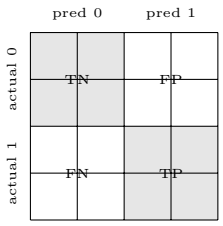
The AUC is relatively close to 1 and greater than 0.5, suggesting some skill in the predictions.

```
seed = 7
kfold = model_selection.KFold(n_splits=10, shuffle=True,
     random_state=seed)
model = LogisticRegression(max_iter=1000)
scoring = 'roc_auc'
results = model_selection.cross_val_score(model, X, Y, cv=
     kfold, scoring=scoring)
print("AUC: %.3f (%.3f)" % (results.mean(), results.std())
     )

# Output:
# AUC: 0.829 (0.047)
```

Confusion Matrix

- The confusion matrix presents the accuracy of a model with two or more classes.
- Each cell counts how often an actual class was predicted as a given class.
- Correct predictions land on the diagonal, every mistake lands off it.
- The two mistakes are not the same: a false positive is a false alarm, a false negative is a miss.



The shaded diagonal holds the correct predictions. Everything off it is an error.

Confusion Matrix

The majority of the predictions fall on the diagonal line of the matrix (which are correct predictions).

```
from sklearn.metrics import confusion_matrix

test_size = 0.33
seed = 7
X_train, X_test, Y_train, Y_test = model_selection.
     train_test_split(X, Y, test_size=test_size,
     random_state=seed)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, Y_train)
predicted = model.predict(X_test)
matrix = confusion_matrix(Y_test, predicted)
print(matrix)

# Output:
# [[142  20]
#  [ 34  58]]
```

Classification Report

- Scikit-learn does provide a convenience report when working on classification problems to give you a quick idea of the accuracy of a model using a number of measures.
- The classification_report() function displays the precision, recall, f1-score and support for each class.

Classification Report

You can see good precision and recall for the algorithm.

```
from sklearn.metrics import classification_report

test_size = 0.33
seed = 7
X_train, X_test, Y_train, Y_test = model_selection.
     train_test_split(X, Y, test_size=test_size,
     random_state=seed)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, Y_train)
predicted = model.predict(X_test)
report = classification_report(Y_test, predicted)
print(report)

# Output:
#               precision    recall  f1-score   support
#          0.0       0.81      0.88      0.84       162
#          1.0       0.74      0.63      0.68        92
#     accuracy                           0.79       254
#    macro avg       0.78      0.75      0.76       254
# weighted avg       0.78      0.79      0.78       254
```

Regression Model Evaluation for Boston House Price Dataset

(Ref: Metrics To Evaluate Machine Learning Algorithms in Python - Jason Brownlee)

Read Data

Import Boston House Price dataset and split into Features (X) and target (Y)

```
import pandas

url = "https://raw.githubusercontent.com/jbrownlee/
     Datasets/master/housing.data"
names = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE'
     , 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT', '
     MEDV']
dataframe = pandas.read_csv(url, sep='\s+', names=names)
array = dataframe.values
X = array[:,0:13]
Y = array[:,13]
```

Mean Absolute Error

- The Mean Absolute Error (or MAE) is the sum of the absolute differences between predictions and actual values. It gives an idea of how wrong the predictions were.
- The measure gives an idea of the magnitude of the error, but no idea of the direction (e.g. over or under predicting).

Mean Absolute Error

A value of 0 indicates no error or perfect predictions. Like logloss, this metric is inverted by the cross_val_score() function.

```
from sklearn.linear_model import LinearRegression

seed = 7
kfold = model_selection.KFold(n_splits=10, shuffle=True,
     random_state=seed)
model = LinearRegression()
scoring = 'neg_mean_absolute_error'
results = model_selection.cross_val_score(model, X, Y, cv=
     kfold, scoring=scoring)
print("MAE: %.3f (%.3f)" % (results.mean(), results.std())
     )

# Output:
# MAE: -3.387 (0.667)
```

Mean Squared Error

- The Mean Squared Error (or MSE) is much like the mean absolute error in that it provides a gross idea of the magnitude of error.

- Taking the square root of the mean squared error converts the units back to the original units of the output variable and can be meaningful for description and presentation. This is called the Root Mean Squared Error (or RMSE).

Mean Squared Error

This metric too is inverted so that the results are increasing. Remember to take the absolute value before taking the square root if you are interested in calculating the RMSE.

```
seed = 7
kfold = model_selection.KFold(n_splits=10, shuffle=True,
      random_state=seed)
model = LinearRegression()
scoring = 'neg_mean_squared_error'
results = model_selection.cross_val_score(model, X, Y, cv=
      kfold, scoring=scoring)
print("MSE: %.3f (%.3f)" % (results.mean(), results.std())
      )

# Output:
# MSE: -23.747 (11.143)
```

R^2 Metric

- The R^2 (or R Squared) metric provides an indication of the goodness of fit of a set of predictions to the actual values. In statistical literature, this measure is called the coefficient of determination.
- $R^2 = 1$ is a perfect fit, and $R^2 = 0$ means the model does no better than always predicting the mean.
- R^2 is *not* bounded below by 0: a model that fits worse than that mean-only baseline scores negative, which happens easily on an unrepresentative fold.

R^2 Metric

Unlike MAE and MSE above, `'r2'` is not negated by `cross_val_score()`: higher is already better, so this mean can be read directly. Note `shuffle=True` in the `KFold` below. This dataset is stored in a sorted order, so without shuffling the folds are unrepresentative, and the score collapses towards the negative-R^2 case from the previous slide.

```
seed = 7
kfold = model_selection.KFold(n_splits=10, shuffle=True,
      random_state=seed)
model = LinearRegression()
scoring = 'r2'
results = model_selection.cross_val_score(model, X, Y, cv=
      kfold, scoring=scoring)
print("R^2: %.3f (%.3f)" % (results.mean(), results.std())
      )

# Output:
# R^2: 0.718 (0.099)
```

Quick Check: Model Evaluation

Think About It

The Pima diabetes dataset has roughly twice as many non-diabetic (class 0) cases as diabetic (class 1) cases. A model that always predicts class 0, no matter the input, could reach around 65% classification accuracy. Which of the metrics on the earlier slides would immediately expose this model as useless, and why does plain accuracy fail to?

Quick Check: Model Evaluation

Answer

AUC would immediately expose it: a constant-output model can't rank any example above another, so it scores AUC $\approx$ 0.5, no better than random, regardless of class imbalance. The confusion matrix would show the same story directly: an entire empty row or column, since class 1 is never predicted. Accuracy fails here because it only counts how many predictions matched, and on an imbalanced dataset "always predict the majority class" matches a lot of the time by construction: exactly why accuracy is called out on the first slide of this section as "the most misused" metric.

Quick Check: Regression Metrics

Think About It

Two house-price models are each wrong by a total of 10 lakh rupees across 10 houses. Model A is off by 1 lakh on every house. Model B is perfect on 9 houses and off by 10 lakh on the tenth. They have the same MAE. Which model does MSE prefer, and which would you rather deploy?

Quick Check: Regression Metrics

Answer

MSE strongly prefers Model A. Squaring the errors makes one large mistake cost far more than many small ones: Model A scores $10 \times 1^2/10 = 1$, while Model B scores $100/10 = 10$, ten times worse, even though MAE rates them identically. Which model you deploy depends on the cost of being wrong. If one badly mispriced house is a disaster, use MSE and pick A. If all errors cost the same per rupee, MAE is the honest metric and the two really are equivalent. This is why the two are reported together rather than one replacing the other.